Lightweight RPC for Kotlin Multiplatform

Master Thesis

Student: Aleksandr Stupnikov

Actual supervisor: Ilmir Usmanov

Problem Statement and Motivation

Kotlin is used for **network-heavy** software: microservices, client-server apps, distributed systems.

Application code:

- **Business logic** — unique per application, high value
- **Network code** — repetitive across applications, **boilerplate**

Problem: network part is too big

Kotlin network technologies

- Ktor
- gRPC
- Kotlin RPC

The Boilerplate Problem — Ktor

Ktor requires to manually write:

- Routing logic (`get("/api/pizza/{id}")`)
- Serialization / deserialization
- HTTP client calls
- Error handling and status codes

This gives **fine control**, but sometimes it is unnecessary.

The Boilerplate Problem — Existing RPC

gRPC and **Kotlin RPC** reduce boilerplate, but require using a service, they are **RMI**:

```
@Rpc interface PizzaShop {
    suspend fun orderPizza(pizza: Pizza): Receipt
}

class PizzaShopImpl : PizzaShop {
    override suspend fun orderPizza(pizza: Pizza): Receipt { .. }
}

val pizzaShop = client.withService<PizzaShop>()
pizzaShop.orderPizza(Pizza("Pepperoni"))
```

Drawbacks of the service-based approach:

1. Must declare and implement an interface just for remote functions
2. Top-level functions cannot be called remotely
3. Remote calls are invisible at the call site

Goals and Objectives

Goal: Develop an RPC framework for Kotlin Multiplatform that reduces boilerplate compared to existing approaches

Objectives:

1. Prototype RPC framework using **context parameters**
2. Prototype support for distributed objects
3. Implement and test
4. Evaluate by comparing with alternatives

Requirements

1. Enable calling **any** named Kotlin function remotely
2. Distinguish remote functions from local ones **at the type level**
3. Preserve **Kotlin Multiplatform** compatibility (no reflection)

Background — Context Parameters

Kotlin's experimental **context parameters** — implicit function parameters resolved from the enclosing scope.

```
context(logger: Logger)
fun greet(name: String) {
    logger.info("Hello, $name")  // logger is passed implicitly
}

fun main() {
    val logger = Logger()
    context(logger) {
        greet("World")  // no need to pass logger explicitly
    }
}
```

- Declared with `context(...)` before the function
- Resolved **automatically** at the call site — no explicit argument needed
- Provide a way to express **environmental requirements** in the type system

Key Idea — Context Parameters for RPC

Use context parameters to carry **remote execution configuration** implicitly:

```
context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long): Long
```

- `RemoteContext` is resolved **implicitly** from the call site
- Determines **where** the function executes (locally or remotely)
- Describes **how** to execute the function (if it executes remotely)
- Makes remote calls **visible at the type level** — you always see the context parameter in the signature

RemoteContext Implememntation

```
sealed interface RemoteContext<out T : RemoteConfig>
object LocalContext : RemoteContext<Nothing>
class ConfiguredContext<T : RemoteConfig>(val config: T) : RemoteContext<T>
```

- Any `@Remote` function can be called in `LocalContext` — always type-checks
- On the server, functions are invoked in `LocalContext` → original body executes

How It Looks to the User

```
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = lhs * rhs
```

Client call:

```
fun main() = runBlocking {
    context(ConfiguredContext(ServerConfig)) {
        println(multiply(6, 7))       // remote call over HTTP
    }
}
```

Server call (inside internal lib):

```
context(LocalContext) { multiply(6, 7) }
```

What Compiler Plugin Generates

The `@Remote` annotation triggers a **body transformation** via IR plugin:

```
// User writes:
@Remote
context(_: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) = lhs * rhs

// Plugin generates:
context(ctx: RemoteContext<RemoteConfig>)
suspend fun multiply(lhs: Long, rhs: Long) =
    if (ctx is LocalContext) {
        lhs * rhs                          // original body
    } else {
        (ctx as ConfiguredContext<RemoteConfig>)
            .config.client.call<Long>(
                RemoteCall("multiply", arrayOf(lhs, rhs))
            )
    }
```

KMP support

No reflection on KMP → need to gather remote functions metadata statically

The plugin replaces `genCallableMap()` intrinsic calls with metadata collected at compile time:

```
// User writes:
val map = genCallableMap()

// Plugin replaces with:
val map = CallableMap(
    "pkg.multiply" to RemoteCallable(
        returnType  = typeOf<Long>(),
        parameters  = arrayOf(typeOf<Long>(), typeOf<Long>()),
        invokator   = { args -> multiply(args[0] as Long, args[1] as Long) }
    ),
    // ... entry for every @Remote function in the source
)
```

No reflection needed — works on **all KMP targets** (JVM, JS, Native, Wasm).

Remote Classes — Distributed Objects

```
@RemoteSerializable
class Calculator(private var state: Int) {
    @Remote context(_: RemoteContext<RemoteConfig>)
    suspend fun multiply(x: Int): Int { state *= x; return state }

    companion object {
        @Remote context(_: RemoteContext<RemoteConfig>)
        suspend operator fun invoke(init: Int) = Calculator(init)
    }
}
```

```
ServerConfig.runWith {
    val calc = Calculator(5)    // created on server, stub returned
    calc.multiply(6)            // method runs on server, state preserved
    calc.multiply(7)            // state = 5 * 6 * 7 = 210
}
```

Remote Classes — How It Works

Serialization:

- Real instance stored in `RemoteInstancesPool` on the server
- Client receives a **stub** (just ID + URL)
- Stub is a generated subclass

Invocation:

- Method call on stub triggers remote call
- Server looks up real instance by ID
- Executes method on the real object

Garbage Collection:

- Lease-based: client periodically renews leases for held stubs
- When leases expire, server removes instance from pool
- Uses weak references to detect when client drops stubs

Ktor Integration

Framework is **transport-agnostic** by design. `core-ktor` module provides HTTP integration:

```
// Server setup — full access to Ktor ecosystem
embeddedServer(Netty, port = 8080) {
    install(Authentication) {
        basic("auth") { validate { ... } }
    }
    install(KRemote) { callableMap = genCallableMap() }
    routing {
        authenticate("auth") {
            post("/secure") { handleRemoteCall() }
        }
        remote("/call")  // unprotected endpoint
    }
}.start()
```

Authentication, logging, CORS, rate limiting — all standard Ktor features work.

Per-Function Boilerplate Comparison

What code must be written **for each new remote function**:

- gRPC
  - IDL for message and rpc method
  - override in impl class
  - toProto() / fromProto()
- Kotlin RPC
  - signature in `@Rpc` interface
  - override in impl class
- Kotlin Remote
  - add `@Remote` and context parameter

Limitations

- **Optimized for shared codebase** — works better when client and server use the same function
- **No cross-language support** — both sides must be Kotlin
- **Serialization** — remote functions' parameters and return values must be serializable
- **Coroutine context** — not preserved across remote boundary

Summary

1. Developed Kotlin Remote — RPC framework using context parameters as coeffects, eliminating service interface boilerplate
2. All objectives were completed

Future Work

1. **Code slicing** — split artifacts per node using `RemoteConfig` types as labels; only include reachable code in each deployment artifact
2. **Remote lambdas** — make function values serializable for higher-order remote calls
3. **Bidirectional communication** — stream data bidirectionally using Flows like in gRPC
4. **Performance optimization** — skip default parameters
5. **Security hardening** — stub URL validation

Comparison with Alternatives

Criterion	gRPC	Kotlin RPC	Kotlin Remote
Service interface required	Yes (protobuf IDL)	Yes (`@Rpc` interface)	No
Top-level functions	No	No	Yes
Remote call visible at call site	No	No	Yes (context param)
Cross-language	Yes	No	No
Multiplatform (KMP)	Partial	Yes	Yes
Transport	HTTP/2	Pluggable (Ktor default)	Pluggable (Ktor default)
Remote classes / objects	No	No	Yes

Todo App — Kotlin Remote

4 CRUD operations, H2 database, Ktor server

```kotlin
// API.kt — the ENTIRE remote API (4 functions, no interface needed)
@Remote context(_: RemoteContext<ServerConfig>)
suspend fun createTodo(request: CreateTodoRequest): Todo = repository.create(request)

@Remote context(_: RemoteContext<ServerConfig>)
suspend fun updateTodo(id: Long, request: UpdateTodoRequest): Todo = repository.update(id, request)

@Remote context(_: RemoteContext<ServerConfig>)
suspend fun deleteTodo(id: Long) = repository.delete(id)

@Remote context(_: RemoteContext<ServerConfig>)
suspend fun todos(): List<Todo> = repository.readAll()
```

```kotlin
// Client — 4 lines of actual usage code
context(ServerConfig.asContext()) {
    createTodo(CreateTodoRequest("Buy milk"))
    createTodo(CreateTodoRequest("Sell cow"))
    println(todos())
}
```

Todo App — gRPC

```
// todo.proto — IDL file (gRPC requires this)
service TodoService {
    rpc CreateTodo (CreateTodoRequest) returns (TodoResponse);
    rpc UpdateTodo (UpdateTodoRequest) returns (TodoResponse);
    rpc DeleteTodo (DeleteTodoRequest) returns (Empty);
    rpc ListTodos (Empty) returns (TodoListResponse);
}
message TodoResponse { int64 id = 1; string title = 2; bool done = 3; string created_at = 4; }
message CreateTodoRequest { string title = 1; }
message UpdateTodoRequest { int64 id = 1; optional string title = 2; optional bool done = 3; }
message DeleteTodoRequest { int64 id = 1; }
message TodoListResponse { repeated TodoResponse todos = 1; }
```

```
// Server — service implementation class (delegates to repository, like Kotlin Remote)
class TodoServiceImpl : TodoServiceGrpcKt.TodoServiceCoroutineImplBase() {
    override suspend fun createTodo(request: Req): TodoResponse = repository.create(request).toProto()
    override suspend fun updateTodo(request: Req): TodoResponse = repository.update(request).toProto()
    override suspend fun deleteTodo(request: Req): Empty { repository.delete(request.id); return Empty }
    override suspend fun listTodos(request: Empty): TodoListResponse = /* ... */
}
```

Plus: proto-to-domain mapping functions

Todo App — Kotlin RPC

```kotlin
// Service interface (required by kRPC)
@Rpc
interface TodoService {
    suspend fun createTodo(request: CreateTodoRequest): Todo
    suspend fun updateTodo(id: Long, request: UpdateTodoRequest): Todo
    suspend fun deleteTodo(id: Long)
    suspend fun todos(): List<Todo>
}

// Service implementation class (required by kRPC)
class TodoServiceImpl : TodoService {
    override suspend fun createTodo(request: CreateTodoRequest) = repository.create(request)
    override suspend fun updateTodo(id: Long, request: UpdateTodoRequest) = repository.update(id, request)
    override suspend fun deleteTodo(id: Long) = repository.delete(id)
    override suspend fun todos(): List<Todo> = repository.readAll()
}
```

```kotlin
val todoService = client.withService<TodoService>()
todoService.createTodo(CreateTodoRequest("Buy milk"))
```

Client and Server Setup

Client

```
object ServerConfig : RemoteConfig {
    override val client = HttpClient {
        defaultRequest {
            url("http://localhost:8080")
        }
        install(ContentNegotiation) {
            json()
        }
    }.remoteClient(genCallableMap(), "/call")
}

fun main() = runBlocking {
    context(ServerConfig.asContext()) {
        println(multiply(6, 7)) // remote
    }
}
```

Server

```
fun main() {
    embeddedServer(Netty, port = 8080) {
        install(KRemote) {
            callableMap = genCallableMap()
        }
        routing {
            remote("/call")
        }
    }.start(wait = true)
}
```

`genCallableMap()` — compiler intrinsic, replaced at compile time with metadata for all `@Remote` functions

What Functions Can Be Remote?

The framework supports a wide range of function kinds:

- **Top-level functions** — `suspend fun multiply(a: Long, b: Long)`
- **Extension functions** — `suspend fun Long.times(rhs: Long)`
- **Class methods** — instance and companion object
- **Nested / local functions** — declared inside other functions
- **Generic functions** — with serializable upper bounds
- **Recursive functions** — recursive calls execute locally
- **Functions that throw** — exceptions propagated transparently

Exception Handling

Remote exceptions are **transparently propagated** to the client:

```
@Remote context(_: RemoteContext<RemoteConfig>)
suspend fun div(x: Int, y: Int) = x / y

fun main() = runBlocking {
    try {
        ServerConfig.runWith { div(10, 0) }
    } catch (e: ArithmeticException) {
        e.printStackTrace()
    }
}
```

```
java.lang.ArithmeticException: / by zero
    at ExceptionKt.div(Exception.kt:11)
    at == Remote Call Boundary ==.(Unknown Source)    ← synthetic frame
    at ExceptionKt.main(Exception.kt:15)
```

Context Hierarchy — Server Capabilities

Contexts form a **subtype hierarchy** expressing server capabilities:

```
interface ArithmeticConfig : RemoteConfig        // basic math
interface TrigonometricConfig : ArithmeticConfig // extends with trigonometry

@Remote context(_: RemoteContext<ArithmeticConfig>)
suspend infix fun Double.mul(rhs: Double) = this * rhs

@Remote context(_: RemoteContext<TrigonometricConfig>)
suspend fun sin(x: Double): Double {
    // can call mul() — ArithmeticConfig is a supertype
    // when on the server: mul() runs locally (LocalContext)
    ...
}
```

Type system enforces which functions are callable from where.

Compiler Plugin — Technical Details

FIR (Frontend)

- Checker: `@Remote` must have `RemoteContext` context param and be `suspend`
- Generator: stub subclass for `@RemoteSerializable`
- Errors visible in IntelliJ IDEA

IR (Backend)

- `RemoteFunctionBodyTransformer` — rewrites bodies
- `CallableMapGenerator` — replaces `genCallableMap()` calls
- `RemoteClassListGenerator` — replaces `genRemoteClassList()` calls